The eXtensible Markup Language (XML)

Background for XML

An Extensible Markup Language (XML) document describes the *structure of data*

Markup language for documents containing structured information

XML and HTML have a similar syntax … both derived from SGML (Standard Generalized Markup Language)

Simple Example

```
<note>
    <to>Tove</to>
    <from>Jani</from>
    <heading>Reminder</heading>
    <body>Don't forget me this weekend!</body>
</note>
```

```
<?xml version="1.0" encoding="UTF-8"?>
- <breakfast_menu>
  - <food>
      <name>Belgian Waffles</name>
      <price>$5.95</price>
      <description>Two of our famous Belgian Waffles with plenty of real maple syrup</description>
      <calories>650</calories>
    </food>
  - <food>
      <name>Strawberry Belgian Waffles</name>
      <price>$7.95</price>
      <description>Light Belgian waffles covered with strawberries and whipped cream</description>
      <calories>900</calories>
    </food>
  - <food>
      <name>Berry-Berry Belgian Waffles</name>
      <price>$8.95</price>
      <description>Light Belgian waffles covered with an assortment of fresh berries and whipped cream</description>
      <calories>900</calories>
    </food>
  - <food>
      <name>French Toast</name>
      <price>$4.50</price>
      <description>Thick slices made from our homemade sourdough bread</description>
      <calories>600</calories>
    </food>
  - <food>
      <name>Homestyle Breakfast</name>
      <price>$6.95</price>
      <description>Two eggs, bacon or sausage, toast, and our ever-popular hash browns</description>
      <calories>950</calories>
    </food>
</breakfast_menu>
```

Main Components of an XML Document

Elements: <Book>

Attributes: <item id="33905">

Entities:

<!ENTITY name "entity_value">

Main Components of an XML Document

- Declaration
- Root Element
- Child Elements
- Attributes
- Text Content

```
<?xml version="1.0" encoding="UTF-8"?>
<library>
  <book id="B101">
    <title>Artificial Intelligence</title>
    <author>John Smith</author>
  </book>
  <book id="B102">
    <title>Data Science</title>
    <author>Sarah Lee</author>
  </book>
</library>
```

The Basic Rules

- XML is case sensitive
- All start tags must have end tags
- Elements must be properly nested
- XML declaration is the first statement
- Every document must contain a root element
- Attribute values must have quotation marks
- Certain characters are reserved for parsing

Document Type Definitions (DTD)

- An XML document may have an optional DTD.
- DTD serves as grammar for the underlying XML document, and it is part of XML language.
- DTD has the form:

  <!DOCTYPE name [markupdeclaration]>

DTD (cont'd)

- DTD for it might be:

```
<!DOCTYPE db [
   <!ELEMENT db (person*)>
   <!ELEMENT person (name, age, email)>
   <!ELEMENT name (#PCDATA)>
   <!ELEMENT age (#PCDATA)>
   <!ELEMENT email (#PCDATA)>
 ]>
```

```
<db>
 <person id="p1">
        <name>John Doe</name>
        <age>30</age>
        <email>john@example.com</email>
 </person>
</db>
```

Some Example DTD Declarations

- Example 1: The Empty Element

```
<!ELEMENT Bool EMPTY> <!--DTD declaration of empty element-->
<Bool Value="True"></Bool> or <Bool Value="True"/> <!--Usage with attribute in
  XML file-->
```

- Example 2: Elements with Data

```
<!ELEMENT Month (#PCDATA)> <!--DTD declaration of an element->
<Month>April</Month> <!—Valid usage within XML file-->
<Month>This is a month</Month> <!—Valid usage within XML file-->
<Month> <!—Invalid usage within XML file, can’t have children!-->
<January>Jan</January>
<March>March</March>
</Month>
```

Some Example DTD Declarations

To specify that an element must have a single child element, include the element name within the parenthesis.

```
<!ELEMENT House (Address)> <!—A house has a single address–>
<House> <!—Valid usage within XML file–>
<Address>1345 Preston Ave Charlottesville Va 22903</Address>
</House>
```

An element can have multiple children. A DTD describes multiple children using a *sequence,* or a list of elements separated by commas. The XML file must contain one of each element in the specified order.

```
<!--DTD declaration of an element–>
  <!ELEMENT address (person,street,city, zip)>
  <!ELEMENT person  (#PCDATA)>
  <!ELEMENT street  (#PCDATA)>
  <!ELEMENT city    (#PCDATA)>
  <!ELEMENT zip     (#PCDATA)>
  <!—Valid usage within XML file–>
  <address>
  <person>John Doe</person>
  <street>1234 Preston Ave.</street>
  <city>Charlottesville, Va</city>
  <zip>22903</zip>
  </address>
```

Cautions concerning DTDs

- All element declarations begin with <!ELEMENT and end with >
- The ELEMENT declaration is *case sensitive*
- The programmer must declare all elements within an XML file
- Elements declared with the #PCDATA content model can not have children
- When describing sequences, the XML document must contain exactly those elements in exactly that order.

XML Schema(XSD)

Simple Element

- `<xs:element name="studentName" type="xs:string"/>`
- `<studentName>Esraa Salah</studentName>`

XML Schema(XSD)

Complex Element

```
<xs:element name="student">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="name" type="xs:string"/>
      <xs:element name="id" type="xs:integer"/>
      <xs:element name="grade" type="xs:decimal"
minOccurs="0"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

```
<student>
 <name>Omar</name>
 <id>102</id>
 <grade>88.5</grade>
</student>
```

XML Schema Cont'd

The previous Schema is interpreted like this:

- <xs:element name="note"> defines the element called "note"
- <xs:complexType> the "note" element is a complex type
- <xs:sequence> the complex type is a sequence of elements
- <xs:element name="to" type="xs:string"> the element "to" is of type string (text)
- <xs:element name="from" type="xs:string"> the element "from" is of type string
- <xs:element name="heading" type="xs:string"> the element "heading" is of type string
- <xs:element name="body" type="xs:string"> the element "body" is of type string

Task

Design an **XML Schema** that defines the structure of an XML document representing a **Library** system.
Your XML should include:

- A root element <library> that contains:
- <book> elements
- <member> elements

Each element should have the following structure:

- <book> contains: <title> (string),<author> (string),<price> (decimal)
- <member> contains: <name> (string), <id> (integer), <borrowedBooks> (integer) — optional

Note:

Use xs:complexType and xs:sequence to define element hierarchies.

Converting Relational Database to XML

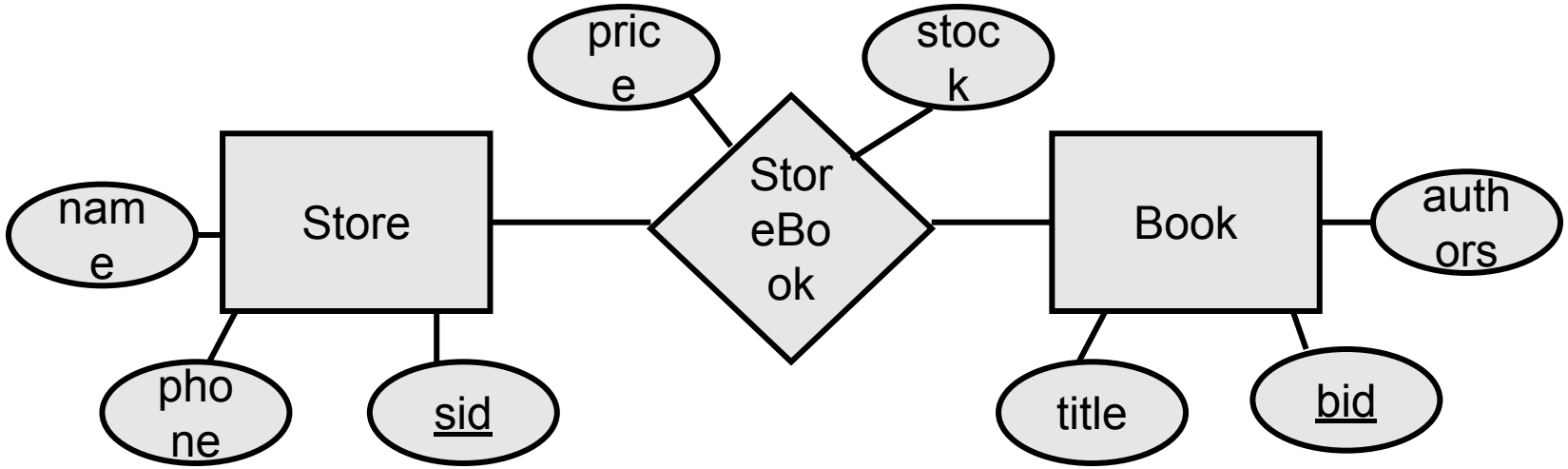
Example: Export the following data into XML and group books by store

- Relational Database:

  Store (sid, name, phone)

  Book (bid, title, authors)

  StoreBook (sid , bid, price, stock)



Converting Relational Database to XML (Cont'd)

- XML:

```
<store> <name> … </name>
        <phone> … </phone>
        <book> <title>… </title>
                <authors> … </authors>
                <price> … </price>
        </book>
        <book>…</book>
        …
</store>
```